

Nx: установка, развёртывание и применение

1. Назначение

Nx — инструмент orchestration для monorepo, ориентированный на project graph, affected execution и task cache.

Позволяет решать следующие задачи:

- построение графа зависимостей;
- запуск задач по нескольким проектам;
- запуск задач только по affected-проектам;
- кэширование результатов задач;
- формализация target model для CI и локальной разработки.

В связке с Flutter monorepo Nx чаще всего используется как инструмент оптимизации исполнения задач, а не как замена Melos.

2. Установка

2.1. Локальная установка

```
npm install --save-dev nx
```

2.2. Установка зависимостей проекта Если Nx уже объявлен в package.json, достаточно выполнить:

```
npm install
```

Глобальная установка Nx не обязательна.

3. Базовая структура конфигурации

Минимально необходимы:

- package.json
- nx.json
- установленные Node dependencies

Для Flutter monorepo Nx можно подключить через plugin, например:

```
{  
  "plugins": [  
    {
```

```

    "plugin": "@nxrocks/nx-flutter",
    "include": ["apps/**/pubspec.yaml", "packages/**/pubspec.yaml"]
  }
]
}

```

Такой подход позволяет:

- автоматически обнаруживать Flutter-проекты;
- строить зависимости по `pubspec.yaml`;
- исключать корневой `pubspec.yaml` из project list.

4. Развёртывание Nx в monorepo

Рекомендуемый порядок:

1. Установить Node.js.
2. Создать `package.json` в корне monorepo.
3. Установить nx как dev dependency.
4. Создать `nx.json`.
5. Подключить Flutter plugin.
6. Установить зависимости через `npm install`.
7. Проверить список проектов.

Проверка:

```
npm nx show projects
```

5. Основные команды

5.1. Список пакетов

```
npm nx show projects
```

5.2. Список affected пакетов:

```
npm nx show projects --affected --base=origin/main --head=HEAD
```

5.3. Граф зависимостей

```
npm nx graph
```

5.4. Очистка кеша и остановка daemon

```
npm nx reset
```

6. Выполнение задач

6.1. Запуск задачи для одного проекта

```
npm nx run simple_calculator:test
```

6.2. Запуск задачи для нескольких проектов

```
npm nx run-many -t test -p simple_calculator simple_magic_8_ball
```

6.3. Запуск задачи для всех проектов

```
npm nx run-many -t build
```

6.4. Запуск нескольких target'ов

```
npm nx run-many -t format,analyze,test
```

6.5. Запуск только по affected-проектам

```
npm nx affected -t test --base=origin/main --head=HEAD
```

6.6. Флаги

- `-t, --target, --targets` - target или список target'ов;
- `-p, --projects` - список проектов или patterns;
- `--exclude` - исключить проекты;
- `--parallel` - степень параллелизма;
- `--output-style` - способ вывода логов;
- `--skipNxCache` - отключить использование cache;
- `--graph` - показать task graph;
- `--nxBail` - остановиться после первой ошибки;
- `--nxIgnoreCycles` - игнорировать циклы в graph.

Примеры:

Параллельный запуск:

```
npm nx run-many -t build --parallel=5
```

Последовательный запуск:

```
npx nx run-many -t test --parallel=false
```

Запуск affected-задачи без cache:

```
npx nx affected -t analyze --base=origin/main --head=HEAD --skipNxCache
```

Статический вывод для CI:

```
npx nx affected -t test --base=origin/main --head=HEAD --outputStyle=static
```

7. Cache

Nx поддерживает cache задач.

Это позволяет:

- не выполнять повторно одинаковые задачи;
- ускорять локальную разработку;
- ускорять CI;
- при использовании remote cache — переиспользовать результаты между раннерами.

Базовые действия:

Очистка cache:

```
npx nx reset
```

Запуск без использования cache:

```
npx nx run-many -t test --skipNxCache
```

8. Task pipelines

Одна из сильных сторон Nx — возможность описывать зависимость задач друг от друга.

Пример концепции:

```
{
  "targetDefaults": {
    "build": {
      "dependsOn": ["^build"]
    }
  }
}
```

Это означает:

- перед запуском `build` текущего проекта сначала должен быть выполнен `build` его зависимостей.

Такой подход особенно полезен для:

- `build pipelines`;
- `code generation`;
- сложных CI workflows.

9. Официальные источники

- Nx Getting Started
- Nx Run Tasks